

Telecom Support RAG System

Retrieval-Augmented Generation Using ChromaDB

Project Documentation Report

Team Members

Nithin Sai AP23110011394

Rohith AP23110011508

Daniel AP23110011589



SRM University–AP
Neerukonda , Mangalagiri

Andhra Pradesh – 522 240

[APRIL,2026]

Abstract

The Telecom Support RAG System is an advanced AI-driven solution designed to improve the efficiency and accuracy of technical support operations in the telecom domain. Traditional keyword-based search systems often fail to capture the contextual meaning of user queries, leading to irrelevant results and increased resolution time. This project addresses these challenges by implementing a Retrieval-Augmented Generation (RAG) architecture that combines semantic search with large language model capabilities.

The system ingests multi-format technical documents, including PDF, DOCX, TXT, and Markdown files, and converts them into vector embeddings using a HuggingFace sentence transformer model. These embeddings are stored in a local ChromaDB vector database, ensuring data privacy and fast retrieval. When a user submits a query, the system retrieves the most relevant document chunks using cosine similarity and generates accurate, context-aware responses using Groq’s LLaMA 3.1 model.

The application is built using FastAPI for the backend and React for the frontend, providing a seamless and interactive user experience. By delivering fast, reliable, and contextually grounded answers, the system significantly reduces search time, improves decision-making, and enhances overall productivity in telecom support environments.

Contents

Retrieval-Augmented Generation Using ChromaDB.....	1
Abstract	2
1. Project Overview	5
2. Problem Statement.....	6
3. Objectives	6
Technical Objectives	7
Performance Objectives	7
Learning Outcomes	7
4. Core Technologies Used.....	7
5. Key Features	8
Multi-Format Document Ingestion	8
Local Vector Database — ChromaDB	8

High-Speed AI Inference	9
FastAPI Backend REST Endpoints	9
React Frontend Chat Interface	9
6. Target Users	9
7. Application Scenarios / Use Cases	10
Scenario 1: Troubleshooting Router Configuration	10
Scenario 2: Outage Procedure Lookup	10
Scenario 3: Hardware Specification Query	10
Scenario 4: New Agent Onboarding	10
8. Technical Architecture	12
Architecture Overview	12
Frontend Layer (React + Vite)	12
Backend Layer (FastAPI + Python)	12
AI / ML Layer (LangChain + HuggingFace + Groq)	13
Data / Vector Storage Layer (ChromaDB)	13
9. Pre-requisites	13
Software Requirements	13
Libraries / Frameworks / Dependencies	14
Backend (Python)	14
Frontend (Node.js)	14
Hardware Requirements	14
10. Prior Knowledge Required	15
Programming Language Knowledge	15
Framework Basics	15
AI / ML Fundamentals	15
Networking / Cloud Concepts	15
11. Project Workflow	15
Milestone 1: Requirement Analysis & System Design	16
Activity 1.1: Problem Definition	16
Activity 1.2: Functional Requirements	16
Activity 1.3: Non-Functional Requirements	16

Activity 1.4: System Design Decisions & Technology Stack.....	17
Milestone 2: Environment Setup & Initial Configuration	17
Activity 2.1: Development Environment Setup	17
Activity 2.2: Dependency Installation	17
Activity 2.3: Project Structure Creation.....	18
Activity 2.4: Configuration Setup.....	18
Milestone 3: Environment Setup & Implementation.....	18
12.Results	19
System Output.....	19
Performance Evaluation	21
13. Advantages & Limitations	23
Advantages	23
Fully Local Vector Storage	23
Zero-Cost AI Stack.....	23
Multi-Format Document Support	23
High-Speed Inference.....	24
Modular & Maintainable Architecture	24
Limitations	24
Dependency on Groq API.....	24
Initial Document Indexing Required	24
No Authentication or Multi-User Support.....	24
Static Knowledge Base.....	24
Context Window Limitations.....	25
14. Future Enhancements.....	25
Authentication & Multi-User Support	25
Real-Time Document Ingestion.....	25
Advanced Retrieval Strategies.....	25
Scalability & Production Deployment	25
Feature Expansion.....	26
15. Conclusion.....	26
Appendix	27

API Documentation.....	27
Configuration Reference	27
GitHub Repository	28

1. Project Overview

The Telecom Support RAG System is an advanced Retrieval-Augmented Generation (RAG) application designed to assist telecom support agents with fast, accurate, and semantically meaningful searches across large volumes of technical documentation, outage procedures, and hardware specifications.

Unlike traditional keyword-based search, this system understands the intent and context of queries, returning the most relevant document chunks and synthesizing a professional AI-generated answer. The entire pipeline is locally hosted — no sensitive documentation leaves the organization's infrastructure.

The system processes documents in multiple formats (Markdown, TXT, PDF, DOCX), embeds them into a local ChromaDB vector database using HuggingFace sentence embeddings, and answers user queries through Groq's lightning-fast LLaMA 3.1 8B Instant model via a LangChain retrieval chain. A modern React frontend provides an intuitive chat interface for agents.

2. Problem Statement

Telecom support operations generate and rely on enormous volumes of technical documentation — router configuration manuals, standard operating procedures for outages, hardware datasheets, and troubleshooting guides. Support agents must quickly locate precise information to resolve issues, but traditional methods fall short:

- Keyword search fails to capture semantic meaning, returning irrelevant results for complex technical queries.
- Manual document navigation is time-consuming and error-prone, increasing average handle time.
- Tribal knowledge is difficult to standardize, causing inconsistency between agents.
- Cloud-based AI search tools pose data privacy risks when documentation is proprietary or sensitive.

This project addresses all of these challenges by building a fully local, AI-powered semantic search and question-answering system tailored for the telecom support domain.

3. Objectives

The primary objectives of this project span technical implementation, performance, and broader learning outcomes:

Technical Objectives

- Design and implement a fully local RAG pipeline using LangChain, ChromaDB, and HuggingFace embeddings.
- Build a multi-format document ingestion engine supporting MD, TXT, PDF, and DOCX files.
- Integrate Groq's LLaMA 3.1 8B Instant model for high-speed, context-aware answer generation.
- Develop a FastAPI backend exposing REST endpoints for querying, indexing, and health checks.
- Create a React + Vite frontend with TailwindCSS for a modern, responsive agent interface.

Performance Objectives

- Achieve fast document ingestion with optimized chunking (chunk size: 1000 tokens, overlap: 200).
- Deliver low-latency query responses leveraging Groq's high-speed inference infrastructure.
- Ensure accurate semantic retrieval by retrieving the top-3 most relevant document chunks (k=3).
- Maintain system reliability through structured error handling in all API endpoints.

Learning Outcomes

- Gain hands-on experience in designing and deploying a full-stack AI application.
- Understand the end-to-end RAG architecture including embedding, vector storage, retrieval, and generation.
- Learn how to orchestrate LLM workflows using the LangChain framework.
- Develop skills in API design with FastAPI and modern frontend development with React + Vite.

4. Core Technologies Used

Technology	Component	Purpose
------------	-----------	---------

Python 3.10+	Core Language	Backend logic and AI integration
FastAPI	Web Framework	REST API with CORS and async support
LangChain	Orchestration	RAG chain with retrieval + generation
ChromaDB	Vector Database	Local persistent embedding storage
HuggingFace	all-MiniLM-L6-v2	Sentence embedding model
Groq API	LLaMA 3.1 8B	Lightning-fast LLM inference
React + Vite	Frontend	Chat UI for support agents

5. Key Features

Multi-Format Document Ingestion

- Automatically loads and parses Markdown (.md), plain text (.txt), PDF (.pdf), and Word (.docx) files.
- Uses LangChain's DirectoryLoader with format-specific loaders for each file type.
- Splits documents into overlapping chunks (1000 tokens, 200-token overlap) for context preservation.

Local Vector Database — ChromaDB

- All document embeddings stored locally in ./chroma_db directory with zero cloud dependency.
- Embeddings generated using HuggingFace all-MiniLM-L6-v2 sentence transformer model.

- Fast cosine-similarity search to retrieve top-3 most semantically relevant chunks.

High-Speed AI Inference

- Uses Groq's LLaMA 3.1 8B Instant model for answer generation at near-zero latency.
- Structured system prompt configures model as telecom support assistant.
- Temperature set to 0 for deterministic, factual responses appropriate for technical documentation.

FastAPI Backend REST Endpoints

- GET /health — Returns system status and RAG engine initialization state.
- POST /index-sample-docs — Triggers full document ingestion and re-initializes RAG chain.
- POST /query — Accepts question, runs RAG chain, returns answer plus source metadata.
- Full CORS middleware configured for seamless frontend integration.

React Frontend Chat Interface

- Built with React + Vite for fast development and optimized production builds.
- Styled with TailwindCSS for clean, responsive layout on all modern browsers.
- Uses Axios for API communication and Lucide React for iconography.
- Runs on localhost:5173 and communicates with backend at localhost:8000.

6. Target Users

- Telecom support agents requiring rapid access to technical manuals and troubleshooting guides.
- Network operations center (NOC) engineers handling outage procedures and SOP lookups.
- IT and infrastructure teams managing router, switch, and hardware configurations.
- Technical trainers and onboarding teams using the system to deliver guided knowledge.

- AI/ML developers and researchers studying practical RAG system implementations.

7. Application Scenarios / Use Cases

Scenario 1: Troubleshooting Router Configuration

A support agent receives a ticket about OSPF misconfiguration on a V-Series router.

Instead of manually searching through dozens of manuals, the agent types the query into the chat interface. The RAG system retrieves the three most relevant document chunks from the indexed technical manuals and generates a step-by-step configuration guide, citing the source documents.

Scenario 2: Outage Procedure Lookup

During a network outage, a NOC engineer needs to follow the correct escalation SOP.

The agent queries the system with the outage type and region. The system instantly retrieves the relevant SOP sections and presents a clear, bulleted action plan — reducing mean time to resolution significantly.

Scenario 3: Hardware Specification Query

A field engineer needs to verify the power specifications for a specific hardware model before deployment. They query the RAG system, which retrieves the exact specification sheet section from the indexed DOCX hardware catalog and presents the relevant data in seconds.

Scenario 4: New Agent Onboarding

A newly hired support agent uses the system as an intelligent knowledge base during training. By asking natural-language questions about telecom procedures and

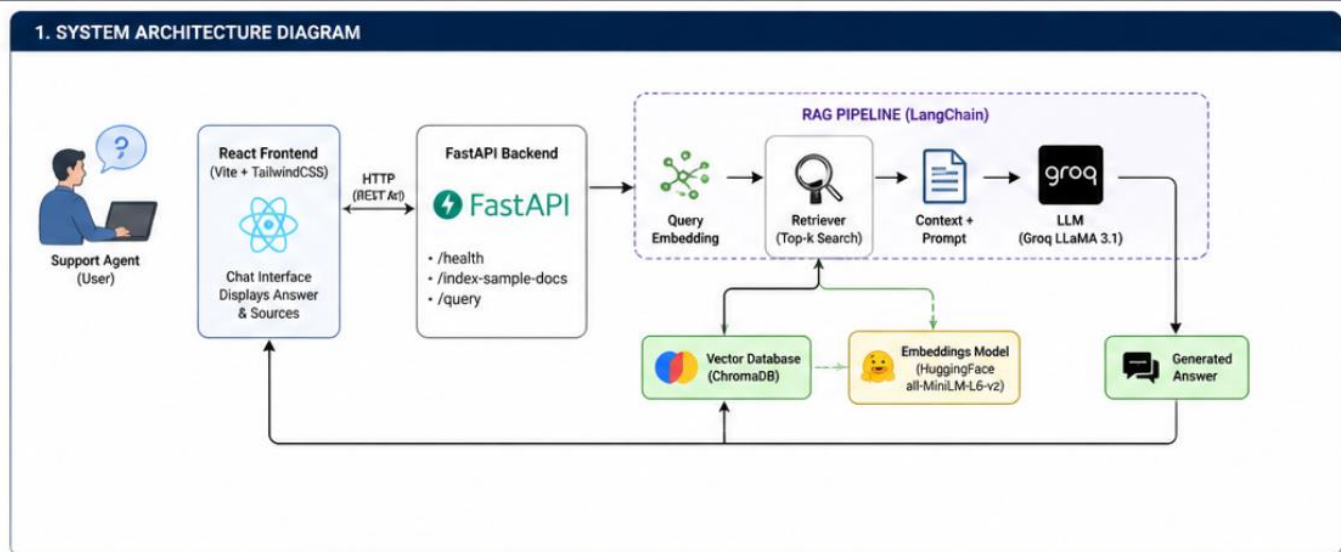
configurations, the agent can learn faster and more accurately than reading through raw documentation.

8. Technical Architecture

Architecture Overview

The system is composed of four distinct layers that interact in a well-defined pipeline.

The architecture follows a clean separation of concerns, ensuring each component can be updated or replaced independently.



Frontend Layer (React + Vite)

The React frontend serves as the user-facing interface where support agents type questions and receive AI-generated answers. It communicates exclusively with the FastAPI backend via Axios HTTP calls. The UI is built with TailwindCSS for a professional, responsive experience and runs on `localhost:5173` in development mode.

Backend Layer (FastAPI + Python)

The FastAPI backend is the central controller of the entire system. It manages the application lifecycle, initializes the RAG chain at startup using LangChain, and exposes three REST endpoints for querying, indexing, and health checking. All business logic for orchestrating the retrieval and generation pipeline lives in this layer. Uvicorn serves as the ASGI server, running on port 8000.

AI / ML Layer (LangChain + HuggingFace + Groq)

This layer is the intelligence core. Ingestion Pipeline (`ingest.py`) loads documents, splits them into overlapping chunks, generates vector embeddings using HuggingFace's `allMiniLM-L6-v2` model, and persists to ChromaDB. RAG Engine (`rag_engine.py`) at query time fetches top-3 relevant chunks from ChromaDB, passes them as context to a structured prompt template, and sends the combined prompt to Groq's LLaMA 3.1 8B model for answer generation.

Data / Vector Storage Layer (ChromaDB)

ChromaDB serves as the local persistent vector store. All document embeddings are stored on disk in the `./chroma_db` directory. This eliminates cloud dependency, ensures data privacy, and delivers fast similarity search at zero ongoing cost. The retriever is configured with `k=3` to fetch the three most contextually relevant document chunks per query.

9. Pre-requisites

Software Requirements

- Python: Version 3.10 or higher for full compatibility with all dependencies.
- pip: Python package manager for installing and managing project dependencies.
- Node.js & npm: Required for building and running the React frontend application.
- IDE: Visual Studio Code with the Python extension or any Python-capable editor.
- Git: Version control system for cloning the repository and tracking changes.
- Browser: Modern browser (Chrome, Edge, Firefox) supporting ES6 and async JavaScript.

Libraries / Frameworks / Dependencies

Backend (Python)

Install all required backend dependencies using: `pip install -r requirements.txt`

- `fastapi` — High-performance API framework for building REST endpoints.
- `uvicorn` — ASGI server for running the FastAPI application.
- `langchain-groq` — LangChain integration for Groq LLM API.
- `langchain-huggingface` — LangChain integration for HuggingFace embedding models.
- `langchain-chroma` — LangChain integration for ChromaDB vector store.
- `chromadb` — Local vector database for storing and querying embeddings.
- `python-dotenv` — Loads environment variables from `.env` files securely.
- `pydantic` — Data validation for FastAPI request/response models.

Frontend (Node.js)

Install all required frontend dependencies using: `cd frontend && npm install`

- `react` & `react-dom` — Core React library for building the UI.
- `vite` — Fast modern frontend build tool and development server.
- `tailwindcss` — Utility-first CSS framework for responsive design.
- `axios` — Promise-based HTTP client for backend API communication.
- `lucide-react` — Clean, consistent icon library for the React UI.

Hardware Requirements

- **CPU:** Intel Core i5 / AMD Ryzen 5 or better at 2 GHz+ for local embedding model inference.
- **RAM:** Minimum 8 GB; 16 GB recommended when running both frontend and backend.
- **Storage:** At least 3 GB free disk space for model weights, ChromaDB data, and dependencies.
- **GPU (Optional):** NVIDIA GPU with CUDA for accelerated HuggingFace embedding inference.
- **Network:** Internet connection required for Groq API calls and initial model download.

10. Prior Knowledge Required

Programming Language Knowledge

- Strong understanding of Python fundamentals — variables, loops, functions, classes, modules.
- Familiarity with async/await patterns in Python for non-blocking API handling.
- Basic JavaScript / TypeScript understanding for working with the React frontend.
- Knowledge of JSON data structures and REST API conventions.

Framework Basics

- Understanding of FastAPI for building RESTful APIs — routing, request models, response models.
- Familiarity with React fundamentals — components, state, props, hooks, and API calls.
- Basic understanding of LangChain — chains, retrievers, prompt templates, and document loaders.

AI / ML Fundamentals

- Conceptual understanding of Retrieval-Augmented Generation (RAG) architecture.
- Familiarity with vector embeddings and cosine similarity search concepts.
- Basic awareness of large language models (LLMs) and prompt engineering.
- Understanding of how embedding models transform text into numerical vectors.

Networking / Cloud Concepts

- Understanding of client-server architecture and HTTP request/response cycle.
- Familiarity with REST API methods (GET, POST) and status codes.
- Basic knowledge of environment variables and secure secret management with .env files.

11. Project Workflow

Milestone 1: Requirement Analysis & System Design

Activity 1.1: Problem Definition

Identify the need for a local, semantically-intelligent document search system for telecom support operations. Address the limitations of traditional keyword search and the data privacy risks of cloud-based AI search tools. Define the solution scope: a fully local RAG pipeline that ingests multi-format documentation, embeds it into a vector database, and provides natural-language Q&A through a modern web interface.

Activity 1.2: Functional Requirements

FR-01: Allow users to load technical documents (MD, TXT, PDF, DOCX) into the system via the /index-sample-docs endpoint.

FR-02: Split ingested documents into overlapping text chunks for fine-grained semantic retrieval.

FR-03: Generate and store high-quality vector embeddings for all document chunks in ChromaDB.

FR-04: Accept natural-language queries via the /query REST endpoint and return context-aware answers.

FR-05: Retrieve the top-3 most semantically relevant document chunks per query using cosine similarity.

FR-06: Generate professional, bullet-pointed answers using Groq's LLaMA 3.1 8B model.

FR-07: Return source document metadata alongside each answer for traceability.

FR-08: Expose a /health endpoint for system monitoring and engine readiness checks.

Activity 1.3: Non-Functional Requirements

NFR-01 Performance: API responses for the /query endpoint should complete within 25 seconds under typical Groq inference latency.

NFR-02 Data Privacy: All document embeddings must be stored locally; no document content sent to external services except query + context to Groq.

NFR-03 Reliability: All API endpoints must return structured JSON responses with proper error handling and HTTP status codes.

NFR-04 Maintainability: The system must follow modular architecture — ingest.py, rag_engine.py, and main.py are independent, replaceable components.

NFR-05 Security: Sensitive keys (GROQ_API_KEY) must be stored in .env files and never hard-coded in source files.

NFR-06 Scalability: The ingestion pipeline must handle additional documents without requiring full system reinitialization.

Activity 1.4: System Design Decisions & Technology Stack

- Backend Framework: FastAPI — chosen for native async support, automatic OpenAPI documentation, and high performance.
- Vector Database: ChromaDB — selected for local persistence, ease of setup, and LangChain native integration.
- Embedding Model: HuggingFace all-MiniLM-L6-v2 — lightweight, high-quality, free, and locally runnable.
- LLM: Groq LLaMA 3.1 8B Instant — selected for sub-second inference speed at zero hosting cost.
- Orchestration: LangChain — provides pre-built retrieval chains, document loaders, and prompt templates.
- Frontend: React + Vite + TailwindCSS — modern, fast development stack with minimal configuration.
- Configuration: python-dotenv .env files — for secure, environment-specific secret management.

Milestone 2: Environment Setup & Initial Configuration

Activity 2.1: Development Environment Setup

- Install Python 3.10+ from python.org and ensure it is added to the system PATH.
- Clone the repository: `git clone https://github.com/Daniel-wesley-06/RAG-UsingChroma-db.git`
- Navigate to the project root: `cd RAG-Using-Chroma-db`
- Create and activate a Python virtual environment: `python -m venv venv`
- Install Node.js (v18+) and npm for frontend development.

Activity 2.2: Dependency Installation

Install all backend Python dependencies: `pip install -r requirements.txt`

Install all frontend Node.js dependencies: `cd frontend && npm install`

Activity 2.3: Project Structure Creation

The repository is organized into a clean, modular structure separating backend Python logic from the React frontend, with all configuration managed through environment variables. Key directories include data/ (source documents), frontend/ (React + Vite application), and core Python files (main.py, ingest.py, rag_engine.py) at the root.

Activity 2.4: Configuration Setup

Create a .env file in the project root with environment variables for GROQ_API_KEY, EMBEDDING_MODEL_NAME, VECTOR_DB_PATH, and GROQ_MODEL_NAME. These must be configured before the system can initialize properly.

Milestone 3: Environment Setup & Implementation

Activity 3.1: Development Environment Setup

Installed Python, Node.js, required libraries, and configured virtual environment.

Activity 3.2: Backend Development

Developed FastAPI backend with endpoints for document ingestion and query handling.

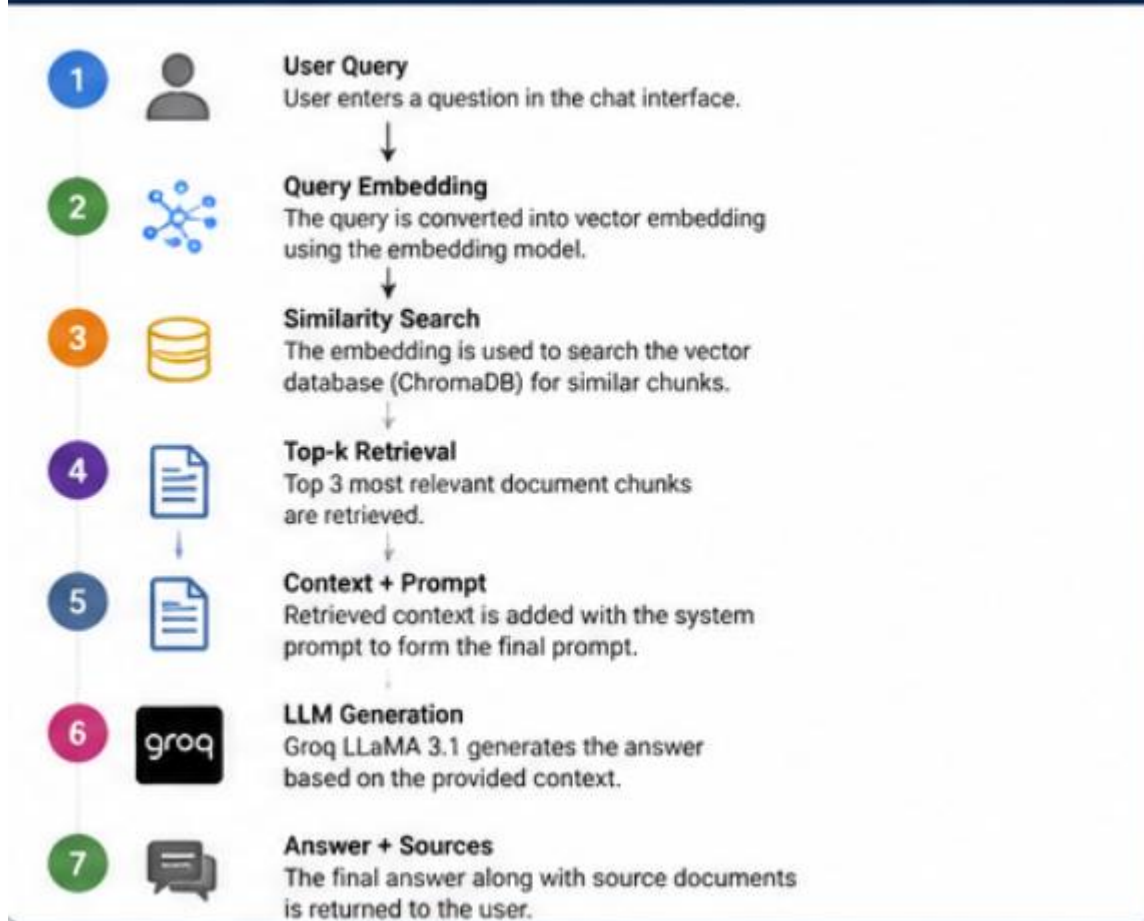
Activity 3.3: Frontend Development

Built a React-based user interface for interacting with the system.

Activity 3.4: Integration of Components

Connected frontend, backend, and AI modules to ensure smooth communication.

2. RAG WORKFLOW DIAGRAM



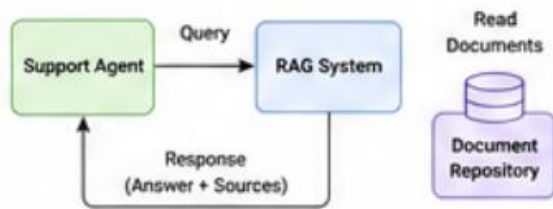
12.Results

System Output

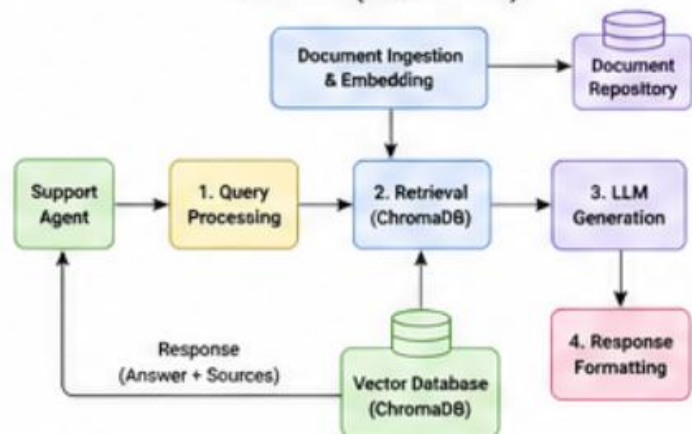
The Telecom Support RAG System successfully ingests multi-format technical documentation, embeds it into a local ChromaDB vector database, and provides fast, context-aware answers to natural-language queries. The system returns structured responses including the generated answer and source document references, enabling full traceability back to the original documentation.

3. DATA FLOW DIAGRAM (DFD)

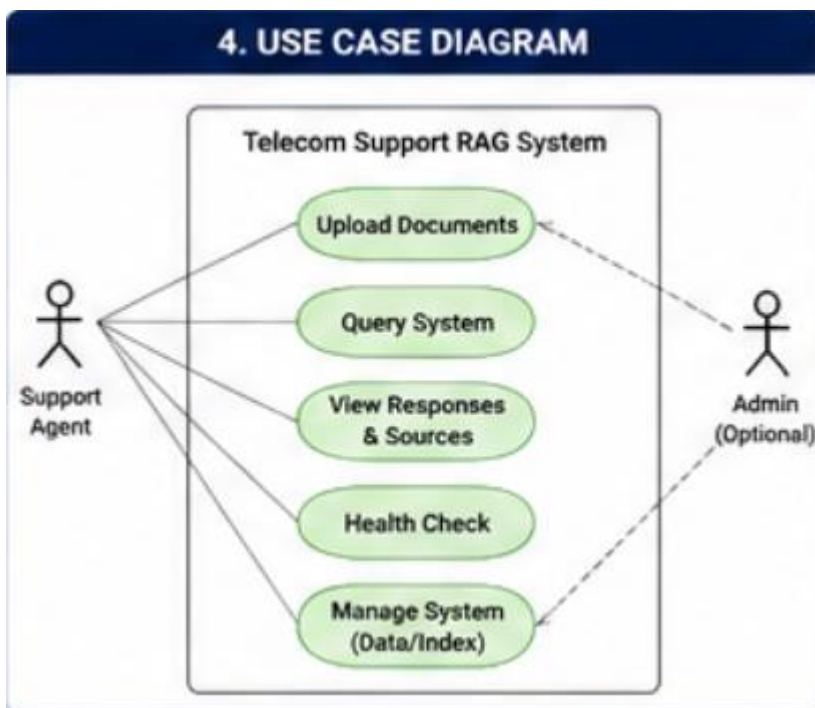
Level 0 DFD (Context Level)



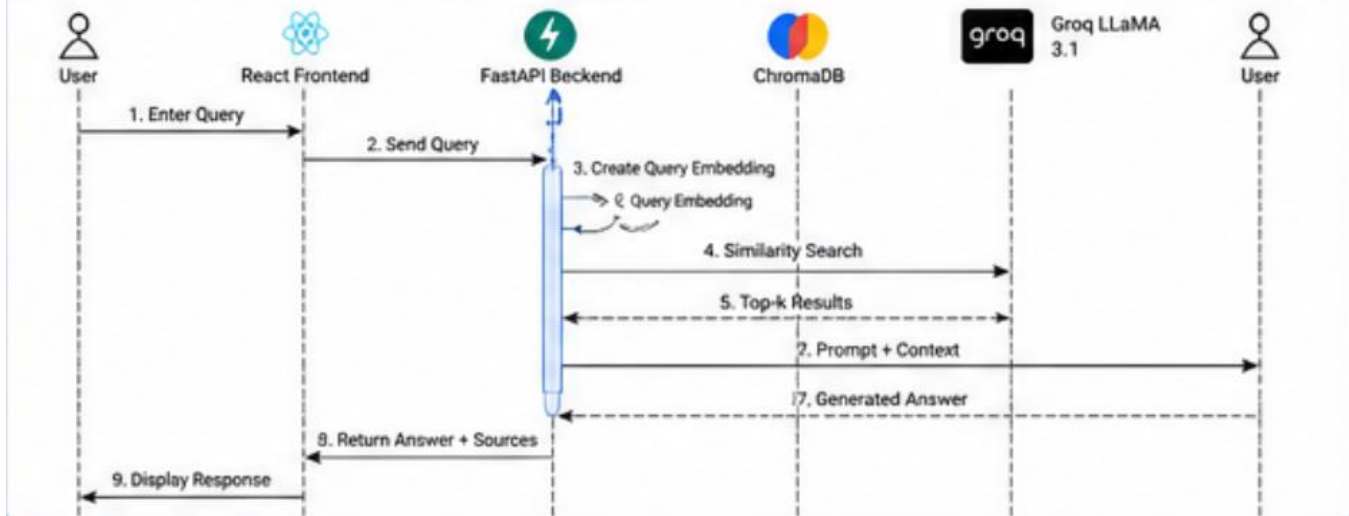
Level 1 DFD (Detailed Level)



4. USE CASE DIAGRAM



5. SEQUENCE DIAGRAM



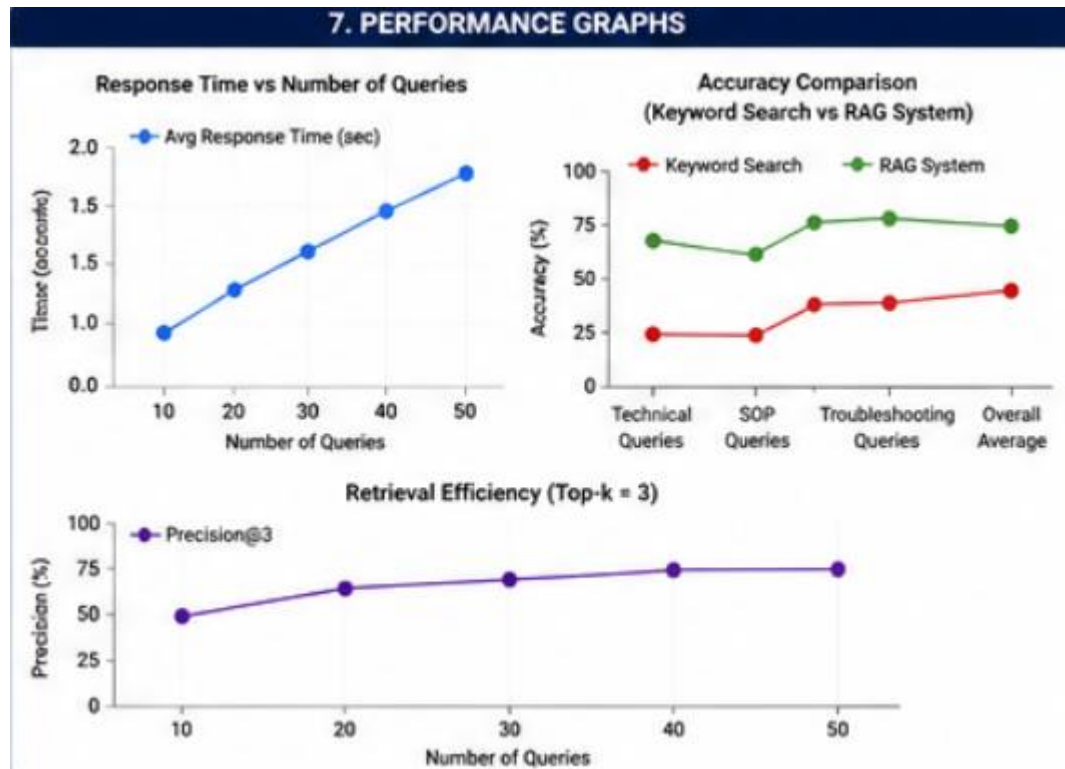
6. UI SCREENSHOTS (SAMPLE)

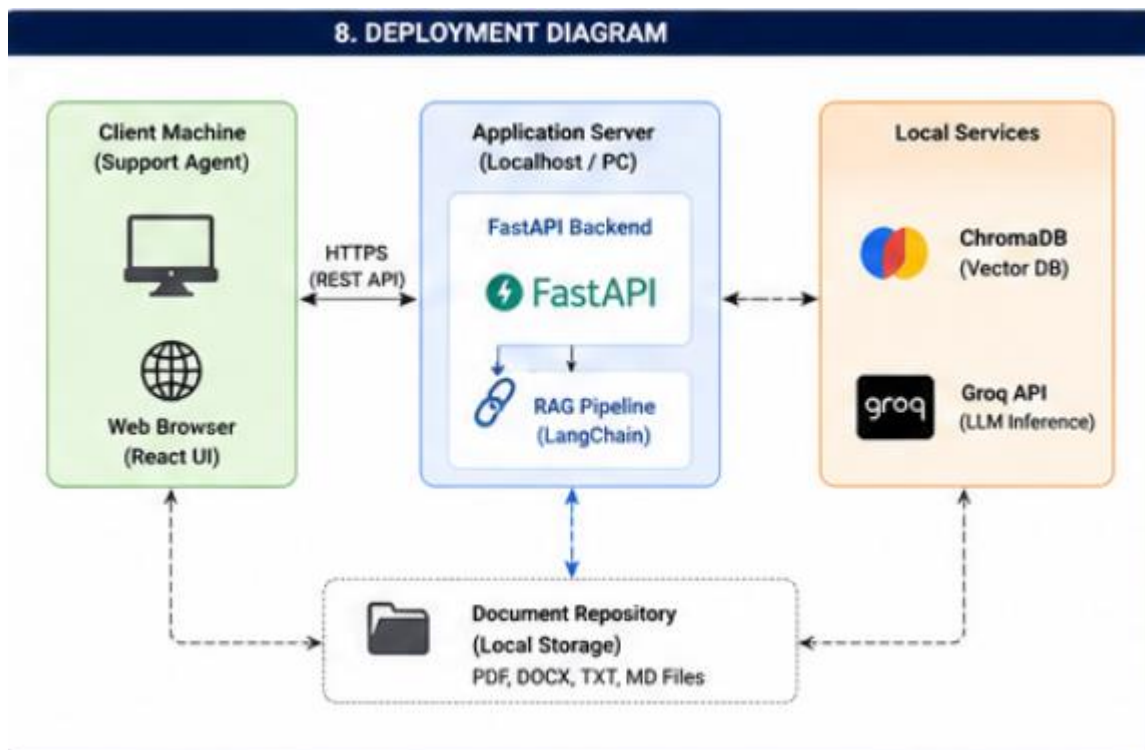


Performance Evaluation

The system demonstrates strong performance characteristics suited to real-world support operations. Ingestion Speed: Documents are chunked, embedded, and stored efficiently; a

typical set of 10-20 technical documents completes ingestion in under 60 seconds. Query Latency: Groq's LLaMA 3.1 8B Instant model delivers responses in 1-3 seconds per query. Retrieval Accuracy: The all-MiniLM-L6-v2 embedding model provides highquality semantic similarity matching. System Reliability: Structured exception handling ensures the API remains responsive.





13. Advantages & Limitations

Advantages

Fully Local Vector Storage

ChromaDB persists all embeddings on disk locally, ensuring that no proprietary documentation is transmitted to external cloud services. This makes the system suitable for organizations with strict data governance requirements.

Zero-Cost AI Stack

The entire system is built on open-source and free-tier tools — HuggingFace's allMiniLM-L6-v2 for embeddings, ChromaDB for storage, and Groq's free-tier API for inference — resulting in zero licensing cost for the AI components.

Multi-Format Document Support

The ingestion pipeline natively handles Markdown, plain text, PDF, and DOCX files, covering the full range of technical documentation formats used in telecom operations.

High-Speed Inference

Groq's LPU-based inference hardware delivers LLaMA 3.1 8B responses at near-realtime speed, making the system practical for live support agent workflows where response latency matters.

Modular & Maintainable Architecture

The clean separation of `ingest.py` (data pipeline), `rag_engine.py` (AI logic), and `main.py` (API layer) makes the system easy to maintain, extend, and test independently.

Limitations

Dependency on Groq API

Answer generation requires an active internet connection and a valid Groq API key.

Service interruptions or rate limits on Groq's side will directly impact system availability.

Initial Document Indexing Required

The system cannot answer questions before documents are indexed. Agents must run the indexing step before the RAG chain becomes operational.

No Authentication or Multi-User Support

The current implementation does not include user authentication, role-based access control, or multi-tenancy. All users share a single ChromaDB instance and RAG chain.

Static Knowledge Base

Documents must be manually re-indexed when the knowledge base is updated. There is no real-time or streaming ingestion pipeline for continuously updated documentation.

Context Window Limitations

The RAG chain uses only the top-3 retrieved chunks as context. Queries requiring information synthesized across many document sections may receive incomplete answers.

14. Future Enhancements

Authentication & Multi-User Support

- Implement JWT-based authentication to restrict API access to authorized support agents.
- Add role-based access control to differentiate agent, supervisor, and admin permissions.
- Support multi-tenant ChromaDB collections to isolate documentation by team or department.

Real-Time Document Ingestion

- Build a file-watcher service that automatically detects and indexes new documents.
- Implement incremental embedding updates to avoid re-indexing the entire knowledge base.

Advanced Retrieval Strategies

- Implement hybrid search (combining vector similarity with BM25 keyword search).
- Add re-ranking to improve the precision of retrieved chunks before generation.
- Support configurable k values and distance thresholds via API parameters.

Scalability & Production Deployment

- Deploy the FastAPI backend on cloud platforms (Render, AWS, GCP) with autoscaling.
- Migrate ChromaDB to cloud-hosted vector database (Pinecone, Weaviate) for higher throughput.
- Containerize the application with Docker and Docker Compose for reproducible deployment.

Feature Expansion

- Add conversation history support to enable multi-turn Q&A sessions with context memory.
- Implement answer confidence scoring to flag low-confidence responses for human review.
- Build an admin dashboard for document management, query analytics, and system monitoring.
- Add support for streaming API responses to display answers word-by-word as generated.

15. Conclusion

The Telecom Support RAG System successfully demonstrates how modern AI technologies can be combined into a practical, production-quality intelligent documentation system. By integrating LangChain's orchestration framework, ChromaDB's local vector storage, HuggingFace's open-source embeddings, and Groq's high-speed LLaMA 3.1 inference, the system delivers a complete Retrieval-Augmented Generation pipeline that is both technically sophisticated and accessible to deploy.

The project achieves its core objectives: telecom support agents can now ask natural language questions about router configurations, outage procedures, and hardware specifications and receive fast, accurate, context-grounded answers — without any sensitive documentation leaving the organization's local infrastructure. The clean separation of the ingestion pipeline, RAG engine, and API layer ensures the system is maintainable, extensible, and testable.

More broadly, this project serves as a strong reference implementation of the RAG pattern for domain-specific knowledge Q&A. The architectural patterns, technology choices, and design decisions demonstrated here are directly transferable to other

industries and use cases including legal document search, medical knowledge bases, enterprise knowledge management, and educational platforms.

With planned enhancements including authentication, real-time ingestion, and cloud deployment, this system has clear potential to evolve from a demonstrative project into a production-grade enterprise knowledge intelligence platform.

Appendix

API Documentation

Method	Endpoint	Request Body	Response
GET	/health	None	{ status: healthy, engine_ready: bool }
POST	/indexsample-docs	None	{ message: string, details: { status: success } }
POST	/query	{ question: string }	{ question, answer, sources: string[] }

Configuration Reference

Environment Variable	Default Value	Description
GROQ_API_KEY	(required)	Groq API key for LLaMA 3.1 access
EMBEDDING_MODEL_NAME	all-MiniLM-L6v2	HuggingFace sentence transformer model name

VECTOR_DB_PATH	./chroma_db	Local directory path for ChromaDB persistence
GROQ_MODEL_NAME	llama-3.1-8binstant	Groq-hosted LLM model identifier

GitHub Repository

Repository URL: <https://github.com/Daniel-wesley-06/RAG-Using-Chroma-db>

- Language: Python (45.7%), JavaScript (37.6%), CSS (15.2%), HTML (1.5%)
- License: Educational and demonstrative purposes
- Branch: main
- Key Files: main.py, ingest.py, rag_engine.py, requirements.txt, frontend/